

Preparación y *preprocesamiento* de datos.

El 80 % del trabajo en un proyecto de datos. Lo aburrido, lo crítico, lo que separa a un modelo bueno de un desastre.

ASIGNATURA	Estrategia del Dato, ML y DL
TEMA	04 • Preparación y Preprocesamiento
DURACIÓN	120 minutos
SESIÓN	04 / 07

Dos bloques. Una clase.

De los datos crudos al dataset *de oro*.

BLOQUE I • 60 MIN • RECOLECCIÓN

- El 80 % del trabajo: por qué importa
- Fuentes de datos: primarias, secundarias, abiertas
- Estructurados, semi y no estructurados
- Pandas: *Series* y *DataFrames*
- Cargar datos: CSV, Excel, SQL
- Explorar: *head*, *info*, *describe*
- 🧑 Demo: cargamos «GymPro»

BLOQUE II • 60 MIN • LIMPIEZA Y ESCALADO

- Datos faltantes: *MCAR*, *MAR*, *MNAR*
- Imputación: media, mediana, KNN, regresión
- Outliers: *IQR*, *Z-score*, Isolation Forest
- Encoding categórico: One-Hot, Label, Target
- Normalización vs estandarización
- Pipelines y *data leakage*
- Quiz final: ¿qué técnica usarías?

EL SECRETO SUCIO DEL DATA SCIENCE

80%

del tiempo de un proyecto de datos se va en *recolectar, limpiar y transformar*. El 20 % restante es lo que sale en LinkedIn: **los modelos.**

Si los datos están sucios, el mejor modelo del mundo dará basura. *Garbage in, garbage out* — la primera ley de la ciencia de datos.

Recolectar los *datos*.

Sin datos no hay modelo. Sin buenos datos, no hay buen modelo. De dónde vienen, cómo se traen, cómo se miran por primera vez.

Fuentes: *primarias* y *secundarias*.

Una **fuentes** es cualquier repositorio del que sacamos información para entrenar modelos. La materia prima.

PRIMARIAS • LA COCINAS TÚ

Recogidas directamente por ti

Encuestas, entrevistas, sensores, formularios web, telemetría de tu app.

Ej. Google Forms a tus 200 socios del gimnasio.

SECUNDARIAS • LAS COCINÓ OTRO

Recogidas por terceros

Estudios publicados, censos, papers, APIs públicas, datasets de Kaggle.

Ej. bajas de Kaggle un dataset con 50.000 rutinas de gym.

ANALOGÍA • LA COMPRA DEL SUPER

Primarias = tomates de tu huerto. Sabes lo que les has echado, pero te ha costado meses.

Secundarias = tomates del super. Cómodo, pero te fías de la etiqueta.

Internas, externas y *datos abiertos*.

¿El dato vive dentro de tu organización o fuera? Los **datos abiertos** son joyas: gratis, públicos, sin restricciones.

INTERNAS

Tu propia casa

CRM, ERP, BD del producto, ventas, logs.

Calidad alta, contexto alto.

EXTERNAS

Fuera de la empresa

APIs, scraping, datos comprados, datos públicos.

Más volumen, menos control.

SINTÉTICAS

Generadas por IA

Cuando no hay reales o son sensibles.

Útil en salud, banca, casos raros.

● GOOGLE DATASET SEARCH

● AWS OPEN DATA

● MICROSOFT RESEARCH

● UCI ML

● KAGGLE

● OMS

● BANCO MUNDIAL

● INE • DATOS.GOB.ES

Estructurados, semi, *no estructurados*.

Define qué herramientas puedes usar y cuánto trabajo te queda.

	ESTRUCTURADOS	SEMI	NO ESTRUCTURADOS
FORMA	Filas y columnas. Esquema rígido.	Etiquetas/jerarquía sin tabla fija.	Sin esquema. Texto, audio, imagen.
EJEMPLO	Excel, SQL, CSV, parquet.	JSON, XML, logs, MongoDB.	PDFs, fotos, vídeos, emails.
TRABAJO PREVIO	Bajo. Casi listo.	Medio. Aplanas el JSON.	Alto. DL para sacarles jugo.
% DEL MUNDO	~20 %	~10 %	~70 % y subiendo

Pandas: el *Excel con esteroides*.

Dos estructuras: **Series** (una columna) y **DataFrame** (toda la tabla).

SERIES • UNA COLUMNA SOLA

```
import pandas as pd

edades = pd.Series([25, 30, 22], name='Edad')
```

Unidimensional con etiquetas. Un solo tipo de dato. Lista con superpoderes.

DATAFRAME • LA TABLA ENTERA

```
import pandas as pd

data = {'Nombre': ['Alice', 'Bob'],
        'Edad': [25, 30]}
df = pd.DataFrame(data)
```

Bidimensional. Cada columna es una Series. Es *la* herramienta.

REGLA DE BOLSILLO

Una variable sola → Series. Cruzar varias → DataFrame. *El 95 % del tiempo, DataFrame.*

Una línea por *cada formato*.

Pandas tiene `read_*` para casi cualquier fuente. Mismo patrón.

CSV

El universal

```
pd.read_csv('gym.csv')
```

Texto plano, separado por comas.

EXCEL

El de los humanos

```
pd.read_excel('gym.xlsx')
```

El que te manda RRHH.

SQL

El profesional

```
pd.read_sql(query, conn)
```

Conexión a la BD de la empresa.

JSON · API

El de internet

```
pd.read_json(url)
```

APIs REST, scraping.

También `read_parquet`, `read_html`, `read_clipboard`... Casi todo lo que se te ocurra.

HILO CONDUCTOR DE LA CLASE

Conoce a *GymPro*.

Cadena de gimnasios. Quieren predecir **qué socios se darán de baja** el próximo mes (*churn*). Tienen un CSV con sus 8 socios más conflictivos. Lo vamos a limpiar a lo largo de la clase.

	ID	NOMBRE	EDAD	PESO_KG	VISITAS_MES	CUOTA_€	PLAN	BAJA
0	001	Ana	34	62.0	12	39.90	Premium	No
1	002	Beto	28	<i>NaN</i>	2	29.90	Básico	Sí
2	003	Carla	45	78.5	20	39.90	<i>NaN</i>	No
3	004	Diego	<i>NaN</i>	91.0	1	29.90	Básico	Sí
4	005	Eva	31	58.0	847	49.90	Premium	No
5	006	Fer	52	72.0	8	39.90	Premium	No
6	007	Gala	23	340	15	29.90	Básico	No
7	008	Hugo	38	80.0	0	<i>NaN</i>	Premium	Sí

4 *NAN*

Beto sin peso. Carla sin plan. Diego sin edad.
Hugo sin cuota.

2 *OUTLIERS*

Eva: 847 visitas/mes (un mes tiene 30 días).
Gala: 340 kg.

2 *CATEGÓRICAS*

«plan» y «baja» no son números. Hay que codificar.

El *ritual* de las primeras miradas.

Acabas de cargar un CSV. ¿Qué tienes? Tres métodos contestan al 80 % de tus dudas iniciales.

01

df.head()

```
df.head()    # 5 primeras  
df.tail()    # últimas 5
```

Las primeras filas. Para ver que cargó bien.

02

df.info()

```
df.info()  
# tipos, no nulos,  
# memoria
```

Tipos y nulos. *De un vistazo, dónde están los NaN.*

03

df.describe()

```
df.describe()  
# mean, std, min,  
# 25%, 50%, 75%, max
```

Estadísticos. Los *outliers cantanza* en min/max.

OTROS QUE VENDRÁN

df.shape, df.dtypes, df.isnull().sum(), df['plan'].value_counts().

HAGAMOS UN ALTO

Lo que llevamos hasta aquí.

01 Tres fuentes abiertas
Kaggle, UCI, Banco Mundial.

02 Series vs
DataFrame
Una columna vs toda la tabla.

03 Tres comandos de
oro
head(), info(), describe().

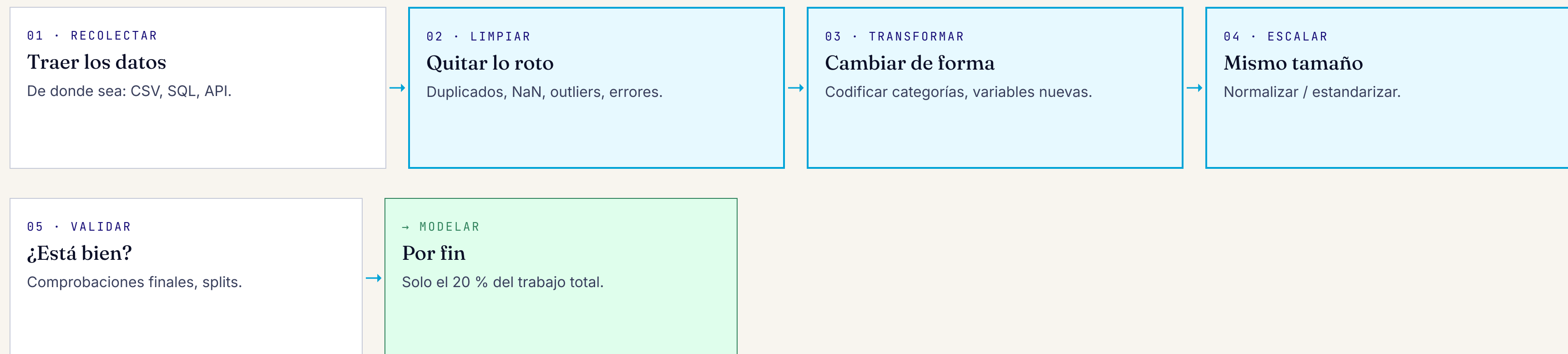
Tenemos los datos. Pero los datos están *guarros*. Toca limpiar.

Limpiar y *transformar.*

Datos faltantes, outliers, escalas dispares, categóricas. Cómo dejar el dataset listo para que el modelo no te insulte.

Pipeline típico. *Cinco* pasos.

Casi cualquier proyecto pasa por estos pasos en este orden. Saltarse uno te muerde después.



Cuidado: es **iterativo**. Vuelves muchas veces. Encuentras un outlier raro → revisas la fuente.

Datos *faltantes*: los huecos.

Un **NaN** (*not a number*) es un agujero en tu dataset. Aparece cuando alguien no rellenó una encuesta, el sensor se apagó, la BD perdió un registro, o se importó mal el archivo.

MCAR • MISSING COMPLETELY AT RANDOM

Falta por azar puro

El sensor se apagó un instante. La hoja se mojó. *No depende de nada*. El caso ideal: puedes ignorarlos sin sesgo.

MAR • MISSING AT RANDOM

Falta según otra variable

«Los hombres dicen menos su peso que las mujeres.» Falta peso pero *depende del género*, que sí tienes. Imputable con cuidado.

MNAR • MISSING NOT AT RANDOM

Falta por la propia variable

«Los que ganan mucho no dicen su sueldo.» El que falta *depende de su propio valor*. El más peligroso: imputar mete sesgo.

	ID	NOMBRE	EDAD	PESO	PLAN	CUOTA
0	001	Ana	34	62	Premium	39.90
1	002	Beto	28	NaN	Básico	29.90
2	003	Carla	45	78	NaN	39.90
3	004	Diego	NaN	91	Básico	29.90
4	005	Eva	31	58	Premium	49.90
7	008	Hugo	38	80	Premium	NaN

```
df.isnull().sum()
# edad      1
# peso      1
# plan      1
# cuota     1
```

¿Sobra? *dropna()*.

La opción más drástica: tirar la fila o la columna. Rápido, pero **pierdes información**. Solo si tienes muchos datos y los que faltan son pocos.

POR FILAS

```
df.dropna(axis='index')
# elimina filas con cualquier NaN

df.dropna(axis='index', how='all')
# solo filas TODO NaN

df.dropna(subset=['edad'])
# solo si falta 'edad'
```

Cuándo: tienes 100k filas y faltan 20.

POR COLUMNAS

```
df.dropna(axis='columns')
# elimina columnas con cualquier NaN

df.dropna(axis='columns', thresh=800)
# solo si tiene <800 valores no nulos
```

Cuándo: una columna tiene el 70 % de NaN, está rota de origen.

CUIDADO · EL SESGO INVISIBLE

Si los NaN no son aleatorios (MNAR), eliminar filas **sesga el dataset**. Ej: los socios cabreados no responden la encuesta → te quedas solo con los felices y tu modelo de churn fracasa.

Rellenar el hueco: *fillna()*.

En vez de tirar la fila, le inventas un valor razonable. Cuatro niveles, de cutre a pro:

NIVEL 1 · CONSTANTE

Un valor fijo

```
df.fillna(0)
```

Cero, «desconocido», «N/A». Solo si sabes lo que haces. Suele meter sesgo brutal.

NIVEL 2 · ESTADÍSTICO

Media / mediana / moda

```
df.fillna(df.mean())  
df.fillna(df.median())
```

Media si es simétrico. *Mediana* si hay sesgo. *Moda* en categóricas.

NIVEL 3 · KNN

El vecino más parecido

```
from sklearn.impute import KNNImputer  
imp = KNNImputer(n_neighbors=5)
```

Buscas las 5 filas más parecidas y promedias su valor para esa columna.

NIVEL 4 · MICE

Modelo predictivo

```
from sklearn.experimental import  
    enable_iterative_imputer  
from sklearn.impute import  
    IterativeImputer
```

Entrena un modelo que predice cada NaN usando las otras columnas. El más caro y preciso.

```
# Simétrica → imputar con media  
# Sesgada → imputar con mediana
```

REGLA PRÁCTICA

La *mediana* es robusta a outliers; la *media* los nota. Cola larga (sueldos, visitas) → mediana. Campana (alturas, edades) → media.

Imputador *en vivo*.

El peso de Beto y la edad de Diego están en NaN. Elige una estrategia y mira cómo cambia. ¿Qué pasa con la **media** y la **desviación**?

ESTRATEGIA DE IMPUTACIÓN

ELIMINAR FILA

MEDIA

MEDIANA

CONSTANTE (0)

```
RESULTADO · COLUMNA peso_kg

n_filas = 8
media = 75.21
mediana = 76.85
desv. est. = 10.37
```

ID	NOMBRE	EDAD	PESO_KG
001	Ana	34	62
002	Beto	28	75.2
003	Carla	45	78.5
004	Diego	35.9	91
005	Eva	31	58
006	Fer	52	72
007	Gala	23	85
008	Hugo	38	80



GRACE M. HOPPER · 1906-1992

Foto: U.S. Navy · dominio público · Wikimedia Commons

// PERSONAJE HISTÓRICO

Grace Hopper.

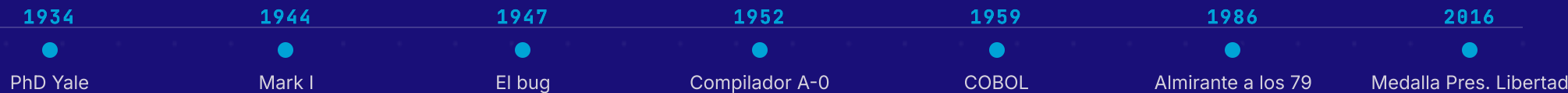
La almirante que enseñó a los ordenadores a hablar como humanos. Y la que, literalmente, encontró el primer *bug*.

Matemática (PhD Yale, 1934) y oficial de la US Navy. Programó el **Mark I** de Harvard en la 2ª Guerra Mundial. En 1952 inventó el primer *compilador* (A-0). Antes de ella, programar era escribir 0s y 1s.

En 1947 una polilla quedó atrapada en un relé del Mark II. Se pegó al cuaderno con la nota «*first actual case of bug being found*». Bug y debug vienen de ahí.

Diseñó **FLOW-MATIC**, abuelo directo de **COBOL** (1959), el lenguaje que aún hoy procesa la mayoría de las transacciones bancarias del mundo.

Se retira como almirante a los 79 años (1986). Medalla Presidencial de la Libertad en 2016. La *Grace Hopper Celebration* es hoy el mayor encuentro mundial de mujeres en tecnología.



«Es más fácil pedir perdón que pedir permiso.»

Outliers: las *ovejas negras*.

Un **outlier** es un valor que se desvía mucho del resto. Puede ser un error (mecanógrafo cansado), una variabilidad natural extrema (Bill Gates entra al bar) o lo más interesante de tus datos (un fraude).

CASO 1 • ERROR

Mecanógrafo cansado

Gala pesa **340 kg**. ¿En serio? Probablemente alguien tecleó 340 en vez de 64.0 o 84. *Hay que arreglarlo o quitarlo.*

CASO 2 • VARIABILIDAD REAL

Eva, la fanática

Eva fue **847 visitas/mes**. Imposible (un mes tiene 30 días). Pero podría ser una atleta de 30 visitas reales. Investiga la fuente.

CASO 3 • LA JOYA

El fraude

En detección de fraude, fugas de información o fallos, los *outliers son lo que buscas*. No los tires: márcalos.

POR QUÉ IMPORTAN

Un solo outlier puede arruinar la **media**, hinchar la **desviación** y descolocar el escalado. Y mete a tu modelo a perseguir un punto que no representa a nadie.

Puntuación Z: «¿cuántas desviaciones te has ido?»

Calculas a cuántas *desviaciones estándar* está cada punto de la media. Si supera **3**, suele considerarse outlier. Funciona bien si los datos siguen una **campana de Gauss**.

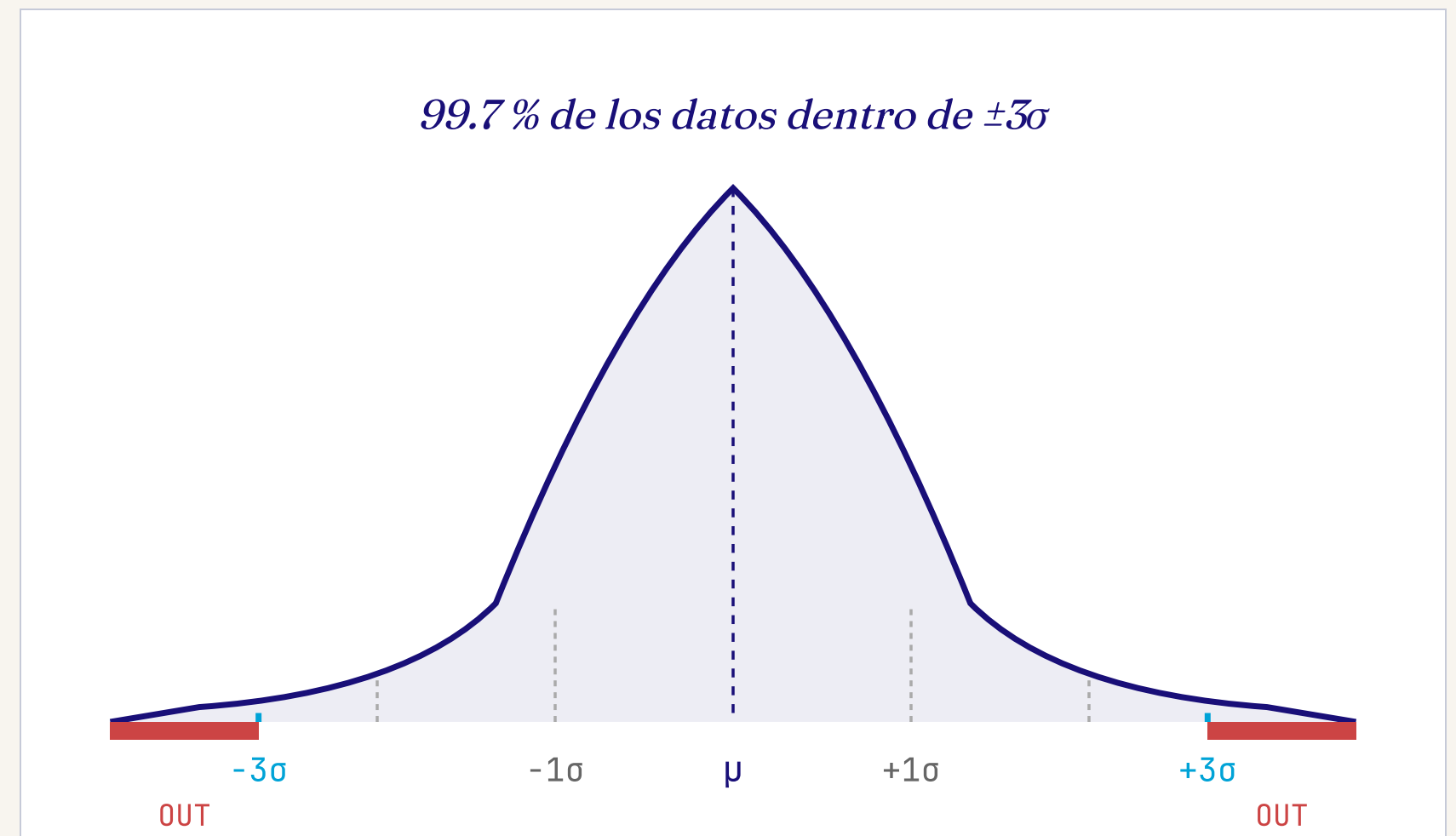
$$z = (x - \mu) / \sigma$$

TRADUCCIÓN

x el valor del punto. **μ** la media de la columna. **σ** la desviación estándar.

Si $|z| > 3 \rightarrow \text{outlier}$.

```
from scipy import stats
z = np.abs(stats.zscore(df['peso']))
outliers = df[z > 3]
```



Limitación: asume distribución normal. Si tus datos están sesgados, usa IQR.

IQR: el *boxplot* hecho regla.

El rango intercuartílico ($Q3 - Q1$) cubre el 50 % central. Fuera de $Q1 - 1.5 \cdot IQR$ o $Q3 + 1.5 \cdot IQR$ es outlier. *No asume ninguna distribución.*

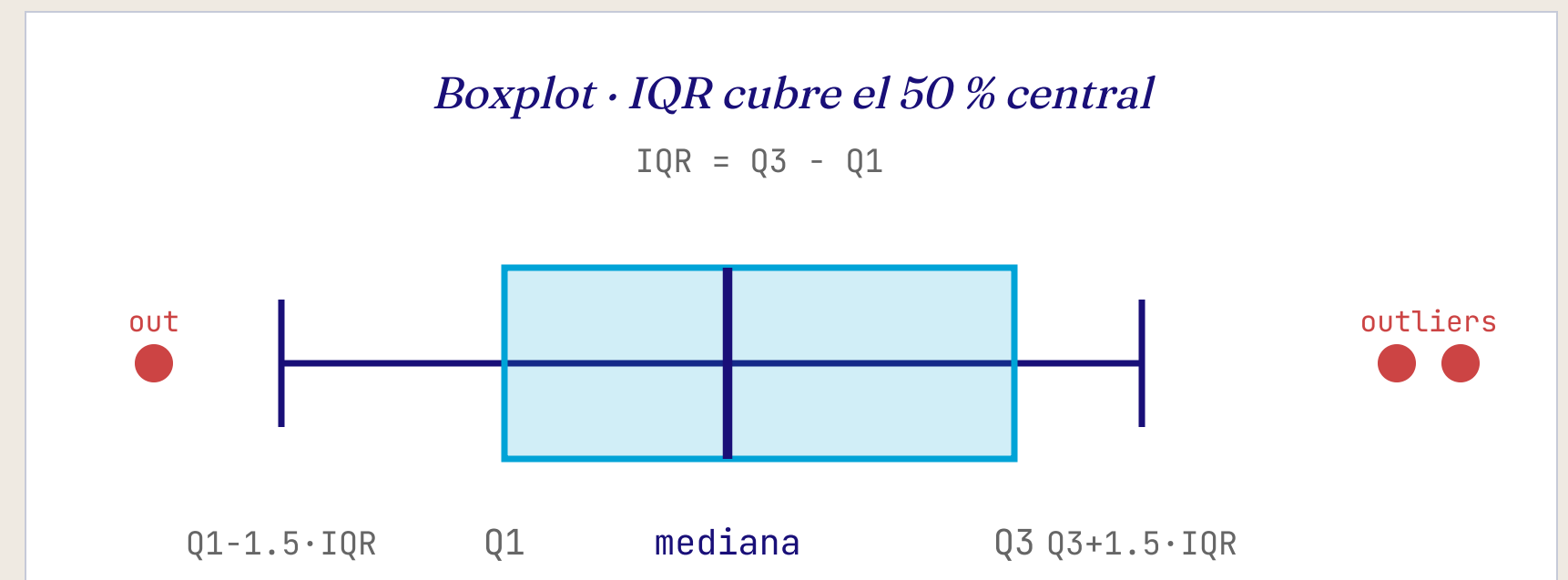
```
Q1 = df['visitas'].quantile(0.25)
Q3 = df['visitas'].quantile(0.75)
IQR = Q3 - Q1
```

```
lim_inf = Q1 - 1.5 * IQR
lim_sup = Q3 + 1.5 * IQR
```

```
outliers = df[
    (df['visitas'] < lim_inf) |
    (df['visitas'] > lim_sup)
]
```

¿POR QUÉ 1.5?

Convención de Tukey, 1977. Cubre el ~99.3 % de una normal. *Una regla, no una verdad.*



Eva (847 visitas) y Gala (340 kg) caerían fuera. Pillados.

Detector IQR *en vivo*.

Mueve el multiplicador. Con 1.5 (default) los dos extremos se marcan. ¿A partir de qué multiplicador deja de pillarlos? ¿Y si bajas a 1.0?

MULTIPLICADOR IQR

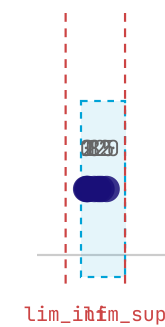
$k \cdot (Q3 + k \cdot IQR)$



1.5

Tukey usó 1.5. Algunos prefieren 3.0 para outliers «extremos».

VISITAS / MES — TODOS LOS SOCIOS



847

datos = [12, 2, 20, 1, 847, 8, 15, 0]

```
visitas_mes · ESTADÍSTICOS
```

```
Q1 = 1.8
```

```
Q3 = 16.3
```

```
IQR = 14.5
```

```
lim_inf = -20.0
```

```
lim_sup = 38.0
```

```
outliers detectados: 1
```

Métodos *multivariantes*.

Z-score y IQR miran **una columna a la vez**. Pero hay outliers que solo se ven al cruzar varias. Ej: «mide 2.10 m» no es raro, «pesa 50 kg» tampoco. *Los dos juntos sí.*

ISOLATION FOREST

El bosque que aísla raros

Crea árboles que parten los datos al azar. Los outliers caen aislados *en pocas ramas* (son fáciles de separar). Eficaz, rápido, escala bien.

```
from sklearn.ensemble import
    IsolationForest
iso = IsolationForest(contamination=0.05)
```

DBSCAN

El que viaja solo

Algoritmo de clustering que considera *ruido* a los puntos sin vecinos cercanos. Encuentra outliers **basándose en densidad**.

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
```

LOF · LOCAL OUTLIER FACTOR

El raro de su barrio

Compara la densidad de un punto con la de sus vecinos. Detecta outliers *locales*, raros incluso dentro de una zona.

```
from sklearn.neighbors import
    LocalOutlierFactor
```

Regla: empieza con IQR/Z. Si el problema lo requiere (datos altamente dimensionales, fraude, anomalías sutiles), pasa a estos.

Encoding de *variables categóricas*.

Los modelos no entienden «Premium». Hay que convertir las categorías a números. Pero **cómo** lo haces afecta. Tres técnicas que tienes que conocer:

LABEL ENCODING

Asigna un entero a cada categoría

PLAN	→	CODE
Básico	→	0
Premium	→	1
VIP	→	2

Cuidado: introduce orden falso (VIP > Premium). Solo si la categoría **sí es ordinal** (S/M/L/XL).

ONE-HOT ENCODING

Una columna por categoría

PLAN	BÁS	PRE	VIP
Básico	1	0	0
Premium	0	1	0
VIP	0	0	1

El estándar. Sin orden falso. *Pero:* con 1.000 categorías → 1.000 columnas. Cuidado con la *cardinalidad*.

TARGET ENCODING

Sustituir por la media del target

PLAN	%BAJA
Básico	0.62
Premium	0.18
VIP	0.08

Cada categoría = media del target en esa categoría. *Riesgo:* data leakage si no se hace dentro de CV.

```
pd.get_dummies(df, columns=['plan']) # one-hot rápido en pandas
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
```

Normalización vs *estandarización*.

Imagina que tu modelo compara **edad** (rango 20–60) con **cuota** (rango 0.30–999.99). La cuota domina por escala, no por importancia. Hay que poner a todas en la misma liga.

NORMALIZACIÓN · MIN-MAX

Llevar todo al rango [0, 1]

$$x' = \frac{(x - \min)}{(\max - \min)}$$

Cuándo: valores entre 0 y 1. Redes neuronales, PCA, KNN.

Pega: los outliers la rompen. Comprimen el resto a casi cero.

ESTANDARIZACIÓN · Z-SCORE

Media 0, desviación 1

$$x' = \frac{(x - \mu)}{\sigma}$$

Cuándo: el default. Regresión, SVM, redes con datos gaussianos.

Bonus: robusta a outliers comparada con Min-Max.

REGLA RAPIDÍSIMA

Si dudas, **StandardScaler**. El default sensato. Min-Max solo si tu algoritmo lo pide (sigmoide, distancias acotadas).

Escalado *en vivo*.

Cuotas mensuales: [29.90, 39.90, 49.90, 39.90, 29.90, 39.90, 99.90]. Cambia el método y mira cómo se transforman los datos. ¿Cuál preserva mejor la forma con el outlier?

MÉTODO DE ESCALADO

SIN TOCAR

STANDARDSCALER

MINMAXSCALER

ROBUSTSCALER

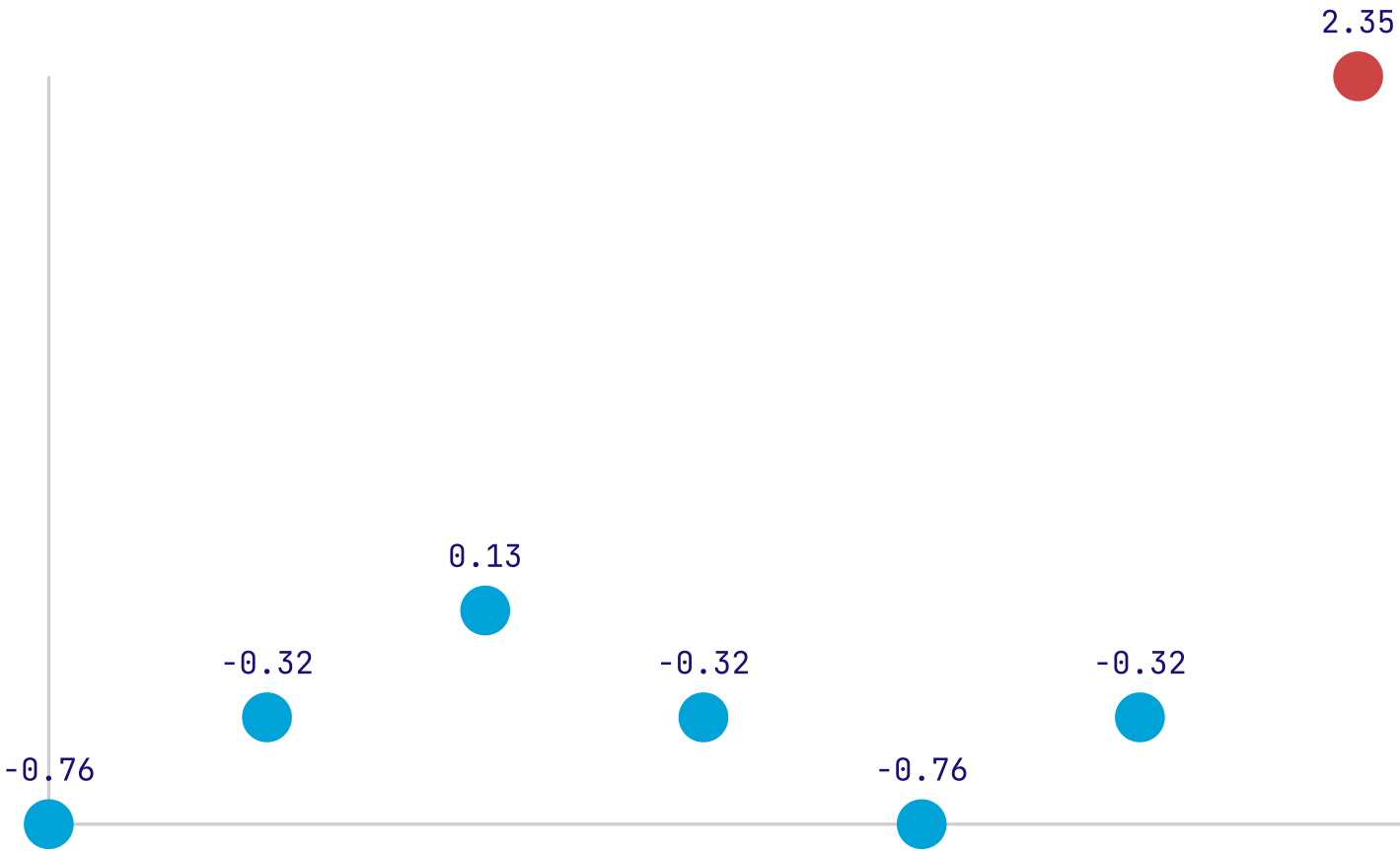
RESULTADO

```
min = -0.76
max = 2.35
media = -0.00
std = 1.00
```

BONUS · ROBUSTSCALER

Usa mediana e IQR en vez de media y std. *Inmune a outliers*. Cuando sospeches que tienes ovejas negras y no quieras quitarlas.

CUOTA_€ · VALORES TRANSFORMADOS



Data leakage y *pipelines*.

El error más común: aplicas el escalado a **todo el dataset antes** de partir en train/test. Información del test se filtra al train. Tu modelo «hace trampa» y falla en producción.

✗ MAL — DATA LEAKAGE

```
# escalas TODO antes de partir → MAL
df_scaled = scaler.fit_transform(df)
X_train, X_test = split(df_scaled)
```

El scaler ha visto el test. Tus métricas estarán infladas.

✓ BIEN — FIT EN TRAIN, TRANSFORM EN TEST

```
X_train, X_test = split(df)
scaler.fit(X_train)      # aprende solo del train
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)  # aplica lo aprendido
```

El test simula *dato nuevo*. Igual que en producción.

PIPELINE · LA FORMA PROFESIONAL

Encadena todo en un solo objeto

```
from sklearn.pipeline import Pipeline; from sklearn.compose import ColumnTransformer
prep = ColumnTransformer([('num', StandardScaler(), ['edad', 'peso']), ('cat', OneHotEncoder(), ['plan'])])
pipe = Pipeline([('prep', prep), ('clf', RandomForestClassifier())]).fit(X_train, y_train)
```

Si te quedas con *cuatro cosas*.

01

El 80 % es preprocesamiento

Modelos brillantes con datos sucios = basura. Acepta que vas a invertir más tiempo limpiando que modelando.

02

NaN: investiga, luego decide

MCAR puedes tirarlos.
MAR/MNAR los imputas con cuidado. La media es tu amiga si la distribución es simétrica; mediana si no.

03

Outliers: IQR primero, luego sofisticado

Z-score si Gauss. IQR para todo lo demás. Si no funcionan, Isolation Forest o LOF. *Y siempre investiga antes de borrar.*

04

Escala con Pipeline, no a mano

StandardScaler por defecto. Fit en train, transform en test. Y siempre, *siempre*, dentro de un Pipeline para evitar leakage.

¿Qué técnica *usarías*?

Tienes una columna «sueldo». La media es 80k pero la mediana es 35k. ¿Cómo imputas los NaN?

MEDIA

MEDIANA

MODA

Tus datos están sesgados (no normales) y necesitas marcar outliers. ¿Qué método eliges?

Z-SCORE

IQR

MEDIA $\pm$ 2

Variable «ciudad» con 4 categorías sin orden, vas a entrenar regresión lineal. ¿Encoding?

LABEL

ONE-HOT

TARGET

Vas a entrenar SVM con variables de escalas muy distintas. ¿Qué escalador?

MINMAX

STANDARD

NINGUNO

Tu columna tiene outliers que no quieres tirar (son reales) pero te molestan al escalar. ¿Qué usas?

MINMAX

STANDARD

ROBUST

Vas a aplicar StandardScaler y luego entrenar. ¿Cómo evitas data leakage?

FIT A TODO

PIPELINE

COPIAR TEST

Lo que tenías que *haber aprendido*.

- Distinguir fuentes **primarias / secundarias**, internas / externas, abiertas
- Cargar datos en pandas con *read_csv, read_excel, read_sql*
- Explorar con *head, info, describe, isnull*
- Tratar NaN: eliminar vs imputar (media, mediana, KNN, MICE)
- Detectar outliers con *Z-score, IQR* e Isolation Forest
- Codificar categóricas con *One-Hot, Label, Target*
- Escalar: normalización *vs* estandarización
- Construir un *Pipeline* sin data leakage

Modelos supervisados de clasificación.

Ya tenemos los datos limpios. Toca por fin entrenar: regresión logística, árboles, random forest, SVM, KNN. Y ver cuál gana en GymPro.

LECTURA SUGERIDA

Pandas + sklearn docs

Pandas «Working with missing data», sklearn «Preprocessing data». Las dos guías oficiales son oro.

PARA PRACTICAR

Reto: limpia GymPro

Coge el CSV, identifica los 4 NaN y 2 outliers, decide estrategia. Comparte tu pipeline en clase la próxima semana.

Datos *limpios.*

El 80 % del trabajo está hecho. Ahora viene la parte fácil: el modelado.

Preguntas, debate o café — en directo o por foro de la asignatura.